🧩 M002 Extension 2 — Piston & Logic Maze

Topic: Two Pistons + AND Conditions · **Course:** Maze Madness · **Level:** Extension (Advanced) · **Time:** about 60 minutes

The agent follows a **redstone trail** through a long maze, using **AND** at some junctions. Twice a **lever powers a piston** to open a gate — drive the agent through the **whole maze**.

Chat words: `l` turn left, `r` turn right, `run` start the solver, `rl` teleport the agent back to you.

1 · Read the Chat Commands 🎛️

Each block binds a **chat word** to an action.

```
def on_chat():
    agent.turn(RIGHT_TURN)
player.on_chat("r", on_chat)
```

What does the agent do when you type `r`?

```
def on_chat():
    agent.teleport_to_player()
player.on_chat("rl", on_chat)
```

You type `rl`. Where does the agent go, and when does that help in a long maze?

This maze has two pistons far apart. Why test the first half with `l` and `r` before you type `run`?

2 · Trace the Solver 🔍

This loop runs when you type `run`. It follows the trail, makes AND decisions, and flips a lever at each piston gate.

```
while True:
    agent.move(FORWARD, 1)
    if agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, AHEAD):
        agent.turn(LEFT_TURN)
        agent.move(FORWARD, 1)
    elif agent.detect(BLOCK, AHEAD) and agent.detect(REDSTONE, RIGHT):
        agent.turn(RIGHT_TURN)
        agent.move(FORWARD, 1)
    elif agent.detect(BLOCK, AHEAD):
        agent.interact(AHEAD)
        agent.move(FORWARD, 1)
```

The loop is `while True`. When does it stop on its own?

First check: `detect(REDSTONE, LEFT) and detect(REDSTONE, AHEAD)`. Describe a spot in the maze where both are true.

The last `elif` is a piston gate: a block ahead, no redstone beside it. What does `agent.interact(AHEAD)` do, and why can the agent move forward right after?

Why does check order matter? What breaks if `elif agent.detect(BLOCK, AHEAD)` ran first, before the two AND checks?

3 · Spot the Bug 🐛

Read each block's goal, rewrite it, then explain the fix.

Bug A — Turn left **only when redstone is left AND ahead**. This turns on either one.

```
if agent.detect(REDSTONE, LEFT) or agent.detect(REDSTONE, AHEAD):
    agent.turn(LEFT_TURN)
    agent.move(FORWARD, 1)
```

Fixed code:

Why was it wrong?

Bug B — The solver should check on **every step** for the whole maze. This checks once and stops.

```
agent.move(FORWARD, 1)
if agent.detect(BLOCK, AHEAD):
    agent.interact(AHEAD)
    agent.move(FORWARD, 1)
```

Fixed code:

Why was it wrong?

Bug C — At a piston gate, **flip the lever first**, then cross the gap. This walks forward before powering the piston, so it hits a closed wall.

```
agent.move(FORWARD, 1)
agent.move(FORWARD, 1)
agent.interact(AHEAD)
```

Fixed code:

Why was it wrong?

4 · Make It Your Own 🛠️

Open the **M002 EXT 2** world and run `run`. **Change one thing** and predict before you test.

Pick **one** (or your own):

- Add a chat word `back` that turns the agent around (two right turns).
- Place a block (`agent.place(...)`) each time it flips a lever, so you can see how far it got.
- Add an `elif` for a junction where redstone is on the **right and ahead** at once.

Which did you pick? Write the code.

Predict: what will happen?

Test it. What actually happened?

5 · Show Your Work 📸 🎥

Get the agent from **start to the very end** — past both piston gates and every junction. Run `run`, see where it sticks, fix, run again until it reaches the goal.

Record **one video** (a phone is fine). Show two things:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Fill the blanks:

> Today I built _____.
>
> I built it using this code: _____.
>
> In this code I used _____.
>
> The hardest part was _____.
>
> That part was hard because _____.

Write your lines here, then say them in your video.

Submit ✅

Send this worksheet + your video to teacher on KakaoTalk.